



Python Fundamentos

Introducción

1



Los estándares de programación se definen comúnmente como un conjunto de reglas, técnicas y prácticas recomendadas que promueven la elaboración de código limpio, legible y eficiente, con errores mínimos. Estos estándares proporcionan un formato coherente para la construcción de programas complejos y altamente funcionales.

La utilización de convenciones adquiere un valor particularmente significativo en proyectos que involucran a más de un programador. La coherencia en la aplicación de convenciones facilita considerablemente la lectura (y comprensión) del código escrito por diferentes personas, contribuyendo así a la eficacia y colaboración en el desarrollo del proyecto.



Adherirse a los estándares de programación otorga innumerables beneficios a un equipo:

- **Incremento en la eficiencia y productividad.** La adhesión generalizada a las buenas prácticas de programación proporciona a todos los miembros del equipo una visión coherente del proyecto en su totalidad.
- **Mejora en la dinámica laboral.** Equipos que adoptan estas pautas exhiben una comunicación más eficaz, comprenden con claridad los roles asignados y logran una ejecución fluida de las tareas.
- **Mejoría en la calidad del producto.** La realización de revisiones entre compañeros y la retroalimentación oportuna permiten detectar problemas en las primeras etapas, posibilitando la implementación de soluciones antes de la producción del producto. Esto se traduce en una reducción de futuros inconvenientes, resultando en un software más resistente y, por ende, en un producto más estable.
- **Facilidad de Mantenimiento.** Se facilita la tarea de mantener el código, ya que cualquier desarrollador puede entenderlo y realizar modificaciones según sea necesario.
- **Optimización de Costos.** Todos los aspectos mencionados anteriormente convergen para generar este beneficio. Un equipo que sigue buenas prácticas minimiza el tiempo dedicado a corregir errores y maximiza la eficiencia en las tareas específicas.

Buenas prácticas

2



- **Refactorización:** Cuando el código se extiende demasiado, como una función que abarca más de una pantalla, se recomienda realizar una refactorización para reducir su tamaño o dividirlo en funciones más pequeñas. Un código más conciso facilita las pruebas y el mantenimiento.
- **Legibilidad:** La legibilidad es más importante que la complejidad. Fragmentos cortos de código complejo y difícil de entender pueden ser problemáticos para el mantenimiento y la modificación del mismo. Es preferible escribir funciones más extensas pero más legibles, respaldadas por documentación clara y pruebas unitarias. Esto facilita la comprensión y colaboración del equipo.
- **README.md:** La creación de una documentación a nivel de proyecto es fundamental para mantener una estructura clara, proporcionando beneficios a todo el equipo en el futuro. Iniciar este proceso en el archivo README.md es un buen punto de partida, con la posibilidad de expandirlo posteriormente en la carpeta docs/ si es necesario. Se deben incluir detalles como el propósito del proyecto, la estructura de cada carpeta, la fuente de los datos, los archivos críticos y las instrucciones para ejecutar pruebas.
- **Requirements.txt:** Un documento de texto donde se listan todas las librerías y dependencias necesarias para un proyecto en Python. Sirve para que otra persona (o tú mismo en otro momento) pueda instalar rápidamente todas las librerías necesarias para que el proyecto funcione.
- **Entornos virtuales:** Crear entornos aislados del sistema operativo simplifica el desarrollo. Entre ellos se incluye el uso de Anaconda o pipenv con Docker, ya que ambas son soluciones adecuadas.



- **Git:** Utilizar una herramienta de control de versiones, como Git, es esencial para evitar la pérdida de cambios críticos. Realizar commits con regularidad y hacer push al repositorio diariamente contribuye a la gestión efectiva del código.
- **.gitignore:** Un archivo especial que le dice a Git qué archivos y carpetas debe ignorar. Esto significa que esos archivos no se subirán al repositorio cuando hagas un commit.
- **Docker:** La utilización de Docker simplifica el despliegue de la aplicación en diferentes máquinas o entornos en la nube. Se trata de una plataforma de código abierto diseñada para facilitar la creación, implementación y ejecución de aplicaciones en contenedores, definiendo estos como entornos portátiles y ligeros que encapsulan la aplicación y sus dependencias.



- **Pruebas de Integración** : En situaciones donde se trabaja con código heredado y se carece de pruebas, es altamente beneficioso incorporar "pruebas de integración". Estas pruebas validan el flujo general del proyecto, asegurando que con ciertos datos de entrada se obtengan los resultados de salida esperados.
- **Docstrings**: Introducir documentación en el código para funciones, clases y módulos es de gran utilidad. Proporcionamos así detalles sobre las expectativas de la función, ofreciendo un ejemplo breve de la salida anticipada. Como fuente de inspiración, se puede revisar el estilo de docstrings utilizado en bibliotecas como Numpy.
- **Código orientado a objetos** : La POO (Programación orientada a objetos) sirve para estructurar el diseño del software en torno a objetos. Al escribir POO, estamos facilitando la reutilización de una parte importante del código para futuros proyectos, evitando además ser redundante en el diseño del mismo.



Cada clase debe hacer una sola cosa y hacerla bien. Si una clase tiene muchas responsabilidades, divídela en clases más pequeñas.

Código con baja cohesión, demasiadas responsabilidades

```
class GestorEmpleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

    def calcular_impuestos(self):
        return self.salario * 0.20 # Cálculo de impuestos

    def generar_reporte(self):
        return f"Empleado: {self.nombre}, Salario: {self.salario}"
```

Código con alta cohesión, separa las responsabilidades

```
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

class CalculadoraImpuestos:
    @staticmethod
    def calcular(empleado):
        return empleado.salario * 0.20

class ReporteEmpleado:
    @staticmethod
    def generar(empleado):
        return f"Empleado: {empleado.nombre}, Salario: {empleado.salario}"
```



El principio DRY (Don't Repeat Yourself) dice que no deberíamos escribir código repetitivo o innecesario. Si algo se repite, debemos optimizarlo, re-utilizarlo o generalizarlo.

Sin el principio DRY

```
class Calculadora:
    def sumar(self, a, b): Method 'sumar' may be 'static'.
        return a + b

    def restar(self, a, b): Method 'restar' may be 'static'.
        return a - b
```

sumar() y **restar()** son funciones independientes, pero requieren una instancia de Calculadora para usarlas, aunque no lo necesitan.

Con el principio DRY

```
class Calculadora2:
    @staticmethod
    def sumar(a, b):
        return a + b

    @staticmethod
    def restar(a, b):
        return a - b
```

Ahora **no necesitamos crear una instancia** para usar estos métodos. Evitamos repetir código innecesario y seguimos **el principio DRY**. Menos código y más eficiencia.



Las Python Enhancement Proposals, o **PEP**, son documentos de diseño que proporcionan información a la comunidad de Python, o describen una nueva característica para Python o sus procesos. Varias PEP son fundamentales para escribir código Python de alta calidad. Entre las más importantes, se encuentra la **PEP8**, que trata sobre el estilo del código Python.

Mantener el estándar de codificación **PEP8** es crucial. Se trata de una guía de estilo que contiene instrucciones y consejos para mantener una escritura de código que sea legible y coherente. Algunas sugerencias importantes aportadas por **PEP8** son:

- **Indentación:** Utiliza 4 espacios por nivel de indentación.
- **Longitud de línea máxima:** No más de 100 caracteres por línea.
- **Espacios en blanco:** Usa espacios alrededor de operadores y después de comas, pero no dentro de paréntesis.
- **Nombres de variables y funciones:** Utiliza `snake\_case` para los nombres de variables y funciones, y `CamelCase` para los nombres de clases.
- **Importaciones:** Realiza importaciones en líneas separadas y agrúpalas en el siguiente orden: módulos estándar, luego bibliotecas de terceros y finalmente locales.



- **Black:** es un formateador de código Python que aplica automáticamente las reglas de **PEP8** y otras convenciones. Utilizando Black, podemos asegurarnos de que el código sigue un estilo uniforme sin tener que formatearlo manualmente, ahorrando energía y tiempo en el proceso de creación. En la imagen de la izquierda vemos un ejemplo de código que ha sido diseñado sin seguir un formato específico. En la imagen de la derecha podemos ver el mismo código después de que se hayan aplicado los criterios de formato con **Black**.

```
def is_unique(  
    s  
    ):  
    s = list(s)  
    )  
    s.sort()  
  
    for i in range(len(s) - 1):  
        if s[i] == s[i + 1]:  
            return 0  
        else:  
            return 1  
  
if __name__ == "__main__":  
    print(  
        is_unique(input())  
    )
```

```
def is_unique(s):  
    s = list(s)  
    s.sort()  
  
    for i in range(len(s) - 1):  
        if s[i] == s[i + 1]:  
            return 0  
    else:  
        return 1  
  
if __name__ == "__main__":  
    print(is_unique(input()))
```



- **Pylint:** es una herramienta que explora el código para encontrar errores y asegurarse de que éste sigue las convenciones de estilo. También puede sugerir mejoras en el código. En las siguientes imágenes vemos un ejemplo sencillo de código que ha sido analizado por Pylint, y que ha devuelto una valoración negativa del mismo.

```
a = 1
b = 2
print(a + b)
```

```
-----
Your code has been rated at -10.00/10
```

Realizando unas pequeñas modificaciones en el código y volviendolo a analizar vemos como la puntuación ha cambiado considerablemente, pues hemos añadido descripciones del funcionamiento y un texto aclaratorio en la invocación del **print()**

```
'''
This program adds two numbers and displays their results
'''
A = 1
B = 2
print('Sum of Numbers:', A + B)
```

```
-----
Your code has been rated at 10.00/10
```



- **Typing:** El type hinting mejora la legibilidad del código, ayuda a los desarrolladores a entender el tipo de datos que se espera y permite herramientas como MyPy verificar tipos estáticos. En Python, esto se consigue utilizando anotaciones de tipos. La librería **Typing** nos permite trabajar con una mayor variedad de tipos de datos, dándonos así la opción de ser más estrictos con nuestro código. La estructura a seguir al declarar una variable es la siguiente:

Por otro lado, un ejemplo sencillo de función creada con type hints sería la siguiente:

```
<nombre de la variable>: <tipo de dato> = <valor>
```

```
def hello_string(s: str) -> None:  
    print(f"Hello {s}")
```



- **MyPy:** es una herramienta para verificar tipos estáticos en Python. En otras palabras, es una manera de asegurar que el código Python sea más preciso y esté libre de errores al identificar problemas de tipo antes de su ejecución en el intérprete de Python. Un ejemplo para que quede más claro el uso de esta herramienta:

```
def suma(a, b):  
    return a + b  
  
resultado = suma(5, "10") # Error de tipo  
print(resultado)
```

Problema: No hay verificación de tipos. Al ejecutar este código, el intérprete lanzará un error en tiempo de ejecución porque no se puede sumar un entero y una cadena de texto.

```
def suma(a: int, b: int) -> int:  
    return a + b
```

Explicación: Ahora, `a` y `b` están anotados como enteros, lo que deja en claro que ambos deben ser de tipo `int`.

```
mypy archivo.py
```

Explicación: Si ejecutamos este comando, MyPy hará que se detecte el problema antes de ejecutar el código y saldrá por pantalla:

```
error: Argument 2 to "suma" has incompatible type "str"; expected "int"
```



- **Isort:** es una herramienta utilizada para ordenar automáticamente las importaciones según las convenciones de Python.

```
import os
import requests
from datetime import datetime
import sys
import json
from collections import defaultdict
```

Problema: Las importaciones no están ordenadas, lo cual puede dificultar la legibilidad y el mantenimiento del código.

```
isort archivo.py
```

Explicación: Isort ordenará las importaciones automáticamente según las convenciones de Python (importaciones estándar, de terceros, y locales).

```
import json
import os
import sys
from collections import defaultdict
from datetime import datetime

import requests
```

Beneficio: Las importaciones quedan ordenadas por tipo y alfabéticamente, lo que mejora la organización y claridad del código.

¡Gracias!